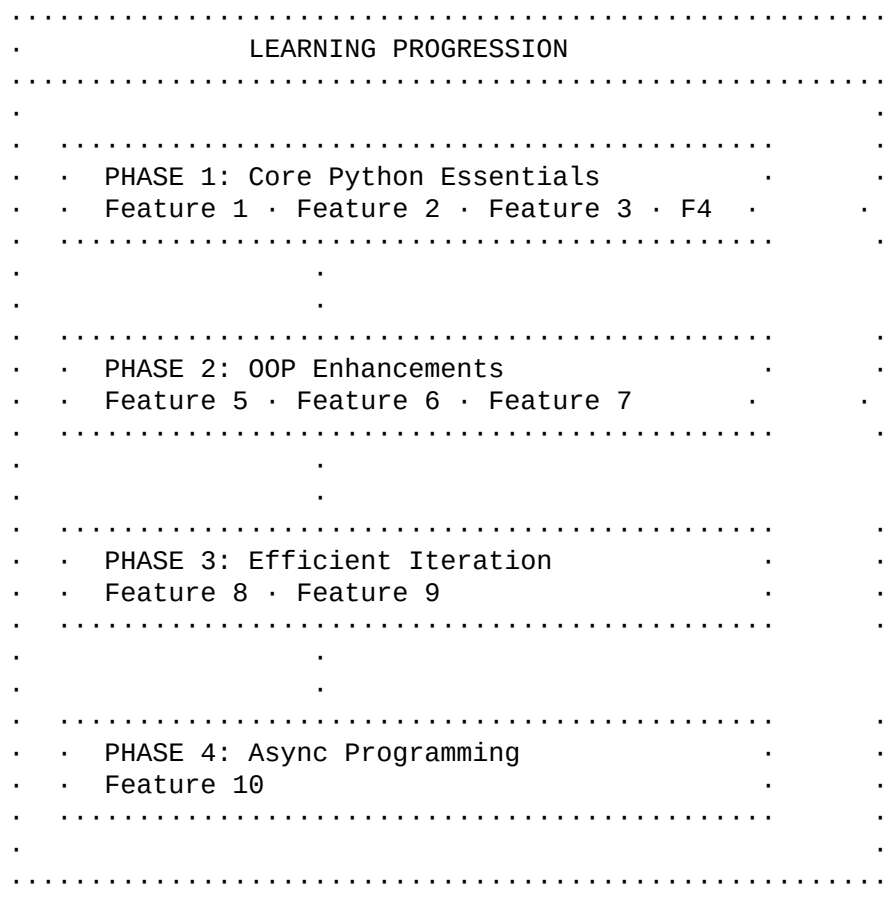
10 Advanced Python Features · Logical Flow

· Project Overview

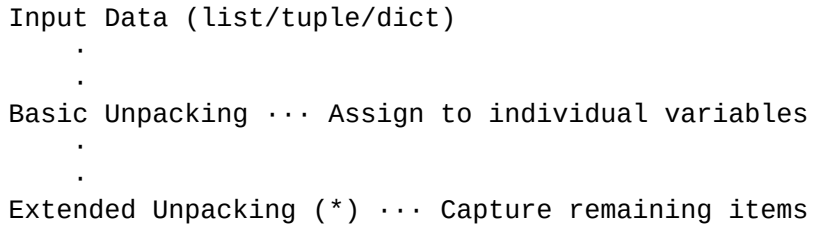
A curated collection of 10 standalone Python scripts demonstrating powerful advanced Python features, progressing from foundational to advanced concepts.

· Overall Learning Flow



· Feature-by-Feature Flow

Feature 1: Unpacking



```
    .
    .
List/Dict Merging ··· Combine collections
    .
    .
Variable Swapping ··· Swap without temp variable
```

Feature 2: exec/eval

```
Code as String
    .
    ···· exec() ··· Execute dynamic code blocks
    .
    ···· eval() ··· Evaluate single expressions safely
```

Feature 3: Type Annotations

```
Define Variables/Functions
    .
    .
Add Type Hints (int, str, List, etc.)
    .
    .
Access __annotations__ at Runtime
    .
    .
IDE/Tool Support & Documentation
```

Feature 4: **repr** & **str**

```
Custom Class Instance
    .
    ···· __repr__() ··· Developer-facing representation
    .
    ···· __str__() ··· User-facing display string
```

Feature 5: Decorators

```
Define Wrapper Function
    .
    .
Apply @decorator Syntax
    .
    .
Original Function ··· Wrapped with extra behavior
    .
    .
Call Decorated Function ··· Pre/Post processing runs
```

Feature 6: Context Managers

```
with FileManager(path) as f:
```

```
    .
    .
__enter__() ··· Acquire resource (open file)
    .
    .
Execute block ··· Use resource
    .
    .
__exit__() ··· Release resource (close file)
```

Feature 7: Custom Iterators

```
MyRange(start, end)
    .
    .
__iter__() ··· Initialize iterator
    .
    .
__next__() ··· Return current value, increment
    .               .
    .               .
    .         StopIteration when end reached
    .
    .
for loop uses protocol automatically
```

Feature 8: Generators

```
count_up_to(max)
    .
    .
yield value ··· Pause execution, return value
    .
    .
next() call ··· Resume from where paused
    .
    .
Memory efficient ··· One value at a time
```

Feature 9: itertools

```
itertools module
    .
    ···· count() ··· Infinite counter
    .
    ···· cycle() ··· Infinite cycling through sequence
    .
    ···· combinations() ··· All possible groupings
```

Feature 10: Async/Await

```
async def main():
```

```
    .
    .
await coroutine_1() ···
await coroutine_2() ····· Concurrent execution
await coroutine_3() ···
    .
    .
asyncio.run(main()) ··· Event loop manages tasks
```